

CONTRACTIQ Sprint 3 – Database Design

Contract Obligation Tracking & Compliance Management Platform

Asvitha Chitteti

1. Purpose of the Database

The ContractIQ database is designed to support the complete contract lifecycle, from contract creation and document versioning to obligation monitoring, renewal management, notifications, reporting, collaboration, and audit tracking. The design separates each responsibility into a dedicated relational table so that information can be maintained without unnecessary duplication.

The schema uses PostgreSQL-compatible data types and a consistent snake_case naming style. Integer identifiers are used for straightforward integration with the existing backend, while timestamps, status fields, and foreign keys provide lifecycle and traceability support.

2. Core Tables

- users – identities, roles, and account status
- contracts – master contract records
- contract_versions – document/version history
- obligations – duties, deadlines, and compliance status
- renewals – renewal planning and decisions
- notifications – alerts and reminders
- reports – generated report records
- audit_logs – traceability of important actions
- activities – operational timeline and collaboration events

3. Proposed Schema

3.1 users

Stores platform users and access-related information.

Column	Data Type	Key / Constraint	Purpose
id	BIGINT	PK	Unique user ID
full_name	VARCHAR(100)		User's full name
email	VARCHAR(255)	UNIQUE	Login/contact email
password_hash	VARCHAR(255)		One-way password hash
role	VARCHAR(40)		admin, manager, legal, reviewer, etc.
is_active	BOOLEAN		Account availability
created_at	TIMESTAMP		Account creation time
updated_at	TIMESTAMP		Last profile update

3.2 contracts

Stores the main business record for every contract.

Column	Data Type	Key / Constraint	Purpose
id	BIGINT	PK	Unique contract ID
owner_id	BIGINT	FK → users.id	Primary user responsible for contract
contract_code	VARCHAR(80)	UNIQUE	Internal contract reference
title	VARCHAR(200)		Contract title
counterparty	VARCHAR(200)		Other organization/person
category	VARCHAR(80)		Service, vendor, NDA, lease, etc.
status	VARCHAR(30)		Draft, active, expired, terminated
risk_level	VARCHAR(20)		Low, medium, high
start_date	DATE		Effective date
end_date	DATE		Expiry date
created_at	TIMESTAMP		Creation timestamp

3.3 contract_versions

Keeps previous document versions and their integrity information.

Column	Data Type	Key / Constraint	Purpose
id	BIGINT	PK	Unique version ID
contract_id	BIGINT	FK → contracts.id	Parent contract
version_no	INTEGER		Sequential version number
document_name	VARCHAR(255)		Uploaded file name
document_uri	TEXT		Controlled storage reference
checksum	VARCHAR(128)		File integrity/deduplication value
change_note	TEXT		Summary of modifications
uploaded_by	BIGINT	FK → users.id	Uploader
uploaded_at	TIMESTAMP		Upload timestamp

3.4 obligations

Tracks contractual duties, ownership, deadlines, priority, and completion.

Column	Data Type	Key / Constraint	Purpose
id	BIGINT	PK	Unique obligation ID
contract_id	BIGINT	FK → contracts.id	Related contract
assigned_to	BIGINT	FK → users.id	Responsible user
title	VARCHAR(200)		Short obligation title
details	TEXT		Full obligation description
due_date	DATE		Completion deadline
frequency	VARCHAR(30)		One-time, monthly, quarterly, annual
priority	VARCHAR(20)		Low, medium, high, critical
status	VARCHAR(30)		Pending, in_progress, completed, overdue
evidence_required	BOOLEAN		Whether proof is required
completed_at	TIMESTAMP		Completion time

3.5 renewals

Manages renewal schedules, notice periods, and renewal decisions.

Column	Data Type	Key / Constraint	Purpose
id	BIGINT	PK	Unique renewal ID
contract_id	BIGINT	FK → contracts.id	Contract being renewed
managed_by	BIGINT	FK → users.id	Responsible user
renewal_date	DATE		Planned renewal date
notice_days	INTEGER		Advance notice requirement
decision	VARCHAR(30)		Pending, renew, decline
new_end_date	DATE		New expiry if renewed
remarks	TEXT		Renewal comments
created_at	TIMESTAMP		Record creation time

3.6 notifications

Stores reminders and alerts delivered to users.

Column	Data Type	Key / Constraint	Purpose
id	BIGINT	PK	Unique notification ID
user_id	BIGINT	FK → users.id	Recipient
contract_id	BIGINT	FK → contracts.id	Related contract
obligation_id	BIGINT	FK → obligations.id	Related obligation, if applicable

renewal_id	BIGINT	FK → renewals.id	Related renewal, if applicable
notification_type	VARCHAR(50)		Deadline, renewal, overdue, system
subject	VARCHAR(200)		Notification heading
message	TEXT		Notification details
is_read	BOOLEAN		Read status
sent_at	TIMESTAMP		Delivery time

3.7 reports

Stores metadata for reports produced from contract and compliance data.

Column	Data Type	Key / Constraint	Purpose
id	BIGINT	PK	Unique report ID
generated_by	BIGINT	FK → users.id	Report creator
contract_id	BIGINT	FK → contracts.id	Optional contract scope
report_type	VARCHAR(50)		Compliance, obligations, renewals, audit
report_name	VARCHAR(200)		Report name
filters	JSONB		Filters used to generate report
file_location	TEXT		Generated file reference
created_at	TIMESTAMP		Generation timestamp

3.8 audit_logs

Records important changes and security-relevant actions.

Column	Data Type	Key / Constraint	Purpose
id	BIGINT	PK	Unique audit event
user_id	BIGINT	FK → users.id	Actor; nullable for system action
contract_id	BIGINT	FK → contracts.id	Related contract, if any
action	VARCHAR(80)		Create, update, delete, login, upload
entity_name	VARCHAR(60)		Affected entity
entity_id	BIGINT		Affected record
before_data	JSONB		Previous state
after_data	JSONB		New state

ip_address	INET		Origin IP, if captured
created_at	TIMESTAMP		Event timestamp

3.9 activities

Provides a timeline for reviews, comments, assignments, uploads, and status changes.

Column	Data Type	Key / Constraint	Purpose
id	BIGINT	PK	Unique activity ID
user_id	BIGINT	FK → users.id	User performing activity
contract_id	BIGINT	FK → contracts.id	Related contract
obligation_id	BIGINT	FK → obligations.id	Related obligation, if applicable
activity_type	VARCHAR(50)		Comment, review, assignment, upload, status_change
description	TEXT		Human-readable activity
metadata	JSONB		Additional structured information
created_at	TIMESTAMP		Activity timestamp

4. Keys and Relationships

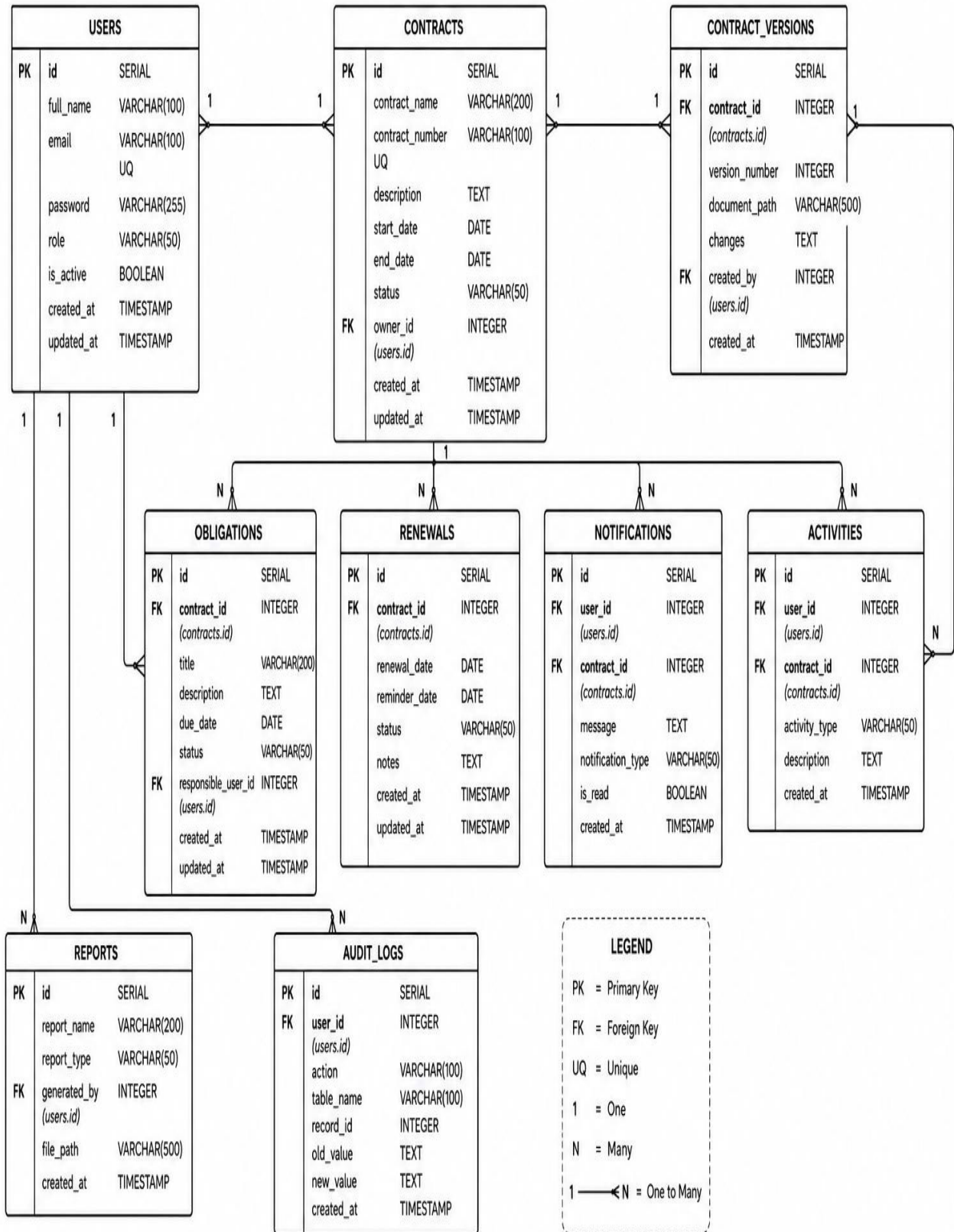
Parent	Child	Cardinality	Foreign-Key Mapping
users	contracts	One to many	users.id → contracts.owner_id
contracts	contract_versions	One to many	contracts.id → contract_versions.contract_id
users	contract_versions	One to many	users.id → contract_versions.uploaded_by
contracts	obligations	One to many	contracts.id → obligations.contract_id
users	obligations	One to many	users.id → obligations.assigned_to
contracts	renewals	One to many	contracts.id → renewals.contract_id
users	renewals	One to many	users.id → renewals.managed_by
users	notifications	One to many	users.id → notifications.user_id
contracts	notifications	One to many	contracts.id → notifications.contract_id

obligations	notifications	One to many	obligations.id → notifications.obligation_id
renewals	notifications	One to many	renewals.id → notifications.renewal_id
users	reports	One to many	users.id → reports.generated_by
contracts	reports	One to many	contracts.id → reports.contract_id
users	audit_logs	One to many	users.id → audit_logs.user_id
contracts	audit_logs	One to many	contracts.id → audit_logs.contract_id
users	activities	One to many	users.id → activities.user_id
contracts	activities	One to many	contracts.id → activities.contract_id
obligations	activities	One to many	obligations.id → activities.obligation_id

5.Entity Relationship Diagram

The diagram below shows the main entities and their one-to-many relationships. The design keeps contracts as the central business entity while users provide ownership, assignment, authorship, and recipient references.

ContractIQ - Database ER Diagram



6.Functional Interaction of the Tables

A user creates or manages a contract, and the contract becomes the parent record for its versions, obligations, renewals, reports, activities, and relevant audit events. Contract versions preserve the evolution of the source documents, while obligations turn contractual requirements into trackable tasks with owners, deadlines, frequency, priority, and evidence requirements.

Renewals handle future lifecycle decisions and can trigger notifications. Reports provide a reusable record of generated compliance information. Activities provide a collaboration timeline, whereas audit logs capture important state changes and security-sensitive actions. Foreign keys connect these modules while keeping each table focused on one responsibility.

7.Constraints and Design Decisions

- contract_code should be unique within the system.
- A contract version number should be unique for a particular contract: UNIQUE(contract_id, version_no).
- Contract end_date should not be earlier than start_date.
- notice_days and other day-count fields should be zero or greater.
- Obligation and renewal status values should be restricted through application validation or CHECK constraints.
- Audit records should be treated as append-only from the application layer.
- Password values must never be stored in plaintext; only password_hash is persisted.
- Foreign-key columns should be indexed because they are frequently used for joins and lookups.
- Document and evidence references should point to controlled storage rather than storing confidential files directly in ordinary database columns.

8.Conclusion

The proposed database design provides a structured, scalable, and efficient foundation for the ContractIQ application. By organizing information into multiple related tables, the design minimizes redundancy while ensuring consistency through relational constraints. The database supports all major functional modules, including contract management, version control, obligation tracking, renewal monitoring, reporting, notifications, auditing, and activity logging. This schema serves as a strong foundation for implementing SQLAlchemy models and backend APIs in the upcoming development sprints.